

Roteiro

- Quais as características e objetivos do OpenMP?
- O que é uma diretiva?
- Como as sentinelas são definidas?
- O que são regiões paralelas?
- Como compilar um programa OpenMP?
- Quais as flags para compilar um programa OpenMP?
- Como definir o número de *threads* que vai executar um programa?
- Diretivas parallel, single e master.

OpenMP

- O OpenMP (Open Multi-Processing) é uma interface de programação de aplicativos (API) que oferece suporte à programação paralela em memória compartilhada.
- É utilizada em múltiplas plataformas, com as linguagens de programação C, C++ e Fortran
- Possui suporte para diversos tipos de processadores e sistemas operacionais, incluindo Solaris, AIX, FreeBSD, HP-UX, Linux, macOS e Windows.
- O padrão consiste de um conjunto de diretivas para compilador, rotinas de bibliotecas e variáveis de ambiente que influenciam um ambiente de execução paralelo baseado em *threads*.

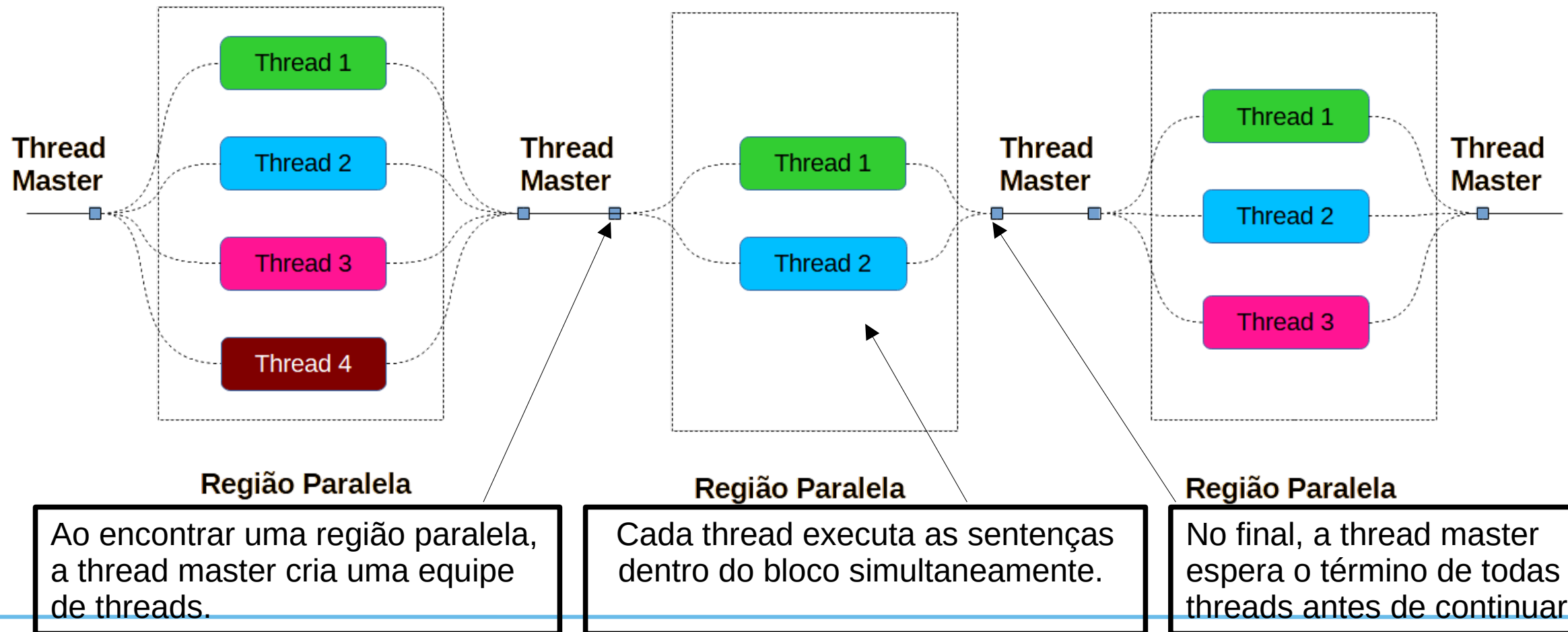
Paralelismo

- O compilador automatiza o processo de tradução de diretivas de alto nível em chamadas para a execução de *threads* que podem executar concorrentemente ou em paralelo.
- O paralelismo no OpenMP é obtido pela execução simultânea de diversas *threads* dentro do que são chamadas de regiões paralelas.
- Haverá ganho real de desempenho se houver processadores disponíveis na arquitetura para efetivamente executar essas regiões em paralelo.
- Em uma das formas de exploração de paralelismo, as diversas iterações de um laço podem ser compartilhadas entre as diversas *threads* e, se não houver dependências de dados entre as iterações do laço, poderão também ser executadas em paralelo.

Regiões Paralelas

- A **região paralela** é a estrutura básica de paralelismo no OpenMP.
- Uma região paralela define uma seção do programa.
- Os programas começam a execução com uma única *thread* (a “*thread master*”).
- Quando a primeira região paralela é encontrada, a *thread master* cria uma equipe de *threads* (modelo fork/join).
- Cada *thread* executa as sentenças que estão dentro da região paralela.
- No final da região paralela, a “*thread master*” espera pelo término das outras *threads* e continua então a execução de outras sentenças.

Modelo Fork/Join



Sintaxe OpenMP

- Um programa em OpenMP nada mais é do que um programa em linguagem C, C++ ou Fortran, acrescido de diretivas e chamadas de funções especiais.
- Os programas em linguagem C ou C++ devem ser iniciados com a inclusão do seguinte cabeçalho, para que as rotinas e macros possam ser reconhecidas pelo compilador:

```
# include <omp.h>
```

- Além disso, opções específicas para cada compilador, devem ser passadas como parâmetros em tempo de compilação, para que as diretivas OpenMP tenham efeito, caso contrário o programa será tratado com um código sequencial comum.

Sintaxe OpenMP

- O OpenMP faz uso conjunto de **diretivas** passadas para o compilador, assim como de algumas funções, para explorar o paralelismo no código em linguagem C ou FORTRAN.
- Uma **diretiva** é uma linha especial de código fonte, com significado especial apenas para o compilador, que se distingue pelo uso de uma **sentinela** no seu início. As sentinelas do OpenMP são:

C/C++:

#pragma omp

Fortran:

!\$OMP (ou C\$OMP ou *\$OMP)

Diretiva Parallel

- A diretiva **parallel** define uma região paralela, ou seja, um trecho de código que será executado pela equipe de *threads* em paralelo. A sintaxe desta diretiva é vista a seguir:

```
#pragma omp parallel [cláusulas]
{
    bloco paralelo
}
```


Exemplo

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        Printf("Alo do OpenMP\n");
    }
    return 0;
}
```

Compilando OpenMP

- Um programa OpenMP pode ser compilado e executado com os seguintes comandos em um terminal de um sistema operacional do tipo Linux utilizando-se o compilador gcc:

```
$ gcc -o teste omp_parallel.c -fopenmp
```

```
$ ./teste
```

- Para os demais compiladores, as flags podem variar um pouco, a tabela a seguir descreve as opções para os compiladores mais comuns.

Resumo das Flags

Compilador	Flag para OpenMP
GCC	-fopenmp
Clang	-fopenmp
Intel (ICC/ICPC)	-qopenmp
MSVC	/openmp
PGI/NVIDIA	-mp
Oracle Studio	-xopenmp

Número de Threads

- `OMP_NUM_THREADS` – especifica o número de *threads* para serem usadas durante a execução de regiões paralelas. Use o comando no Linux:
`$export OMP_NUM_THREADS=4`
- O valor padrão para esta variável é **1**. `OMP_NUM_THREADS` threads serão usadas para executar o programa independente do número de processadores físicos disponíveis no sistema.
- Como resultado, você pode executar programas com mais *threads* do que o número de processadores físicos e eles vão executar corretamente. Contudo, o desempenho da execução dos programas nesta maneira poderá ser ineficiente.

Número de Threads

- Dentro do programa, a cláusula `num_threads(n)` pode ser utilizada em conjunto com as diretivas do OpenMP, que são precedidas sempre de `#pragma omp`.

- Por exemplo:

```
#pragma omp parallel num_threads(8)
```

- Em qualquer ponto do programa, com a chamada da função `omp_set_num_threads(n)`, que tem a seguinte sintaxe em C/C++:

```
#include <omp.h>
```

```
void omp_set_num_threads(int num_threads);
```

Número de Threads

- A precedência segue a seguinte ordem (do mais alto ao mais baixo):
 - O valor especificado na cláusula ``num_threads``
 - A função ``omp_set_num_threads()`` chamada no programa.
 - A variável de ambiente ``OMP_NUM_THREADS``
 - O número padrão de threads da implementação OpenMP que, geralmente, corresponde ao número de núcleos disponíveis, mas pode variar conforme a implementação.

Número de Threads

- Uma das funções mais utilizadas no OpenMP retorna o número de threads que estão sendo utilizadas naquela região paralela.

C/C++:

```
#include <omp.h>  
int omp_get_num_threads(void);
```

- Nota importante: esta função retorna **1** se for chamada fora de uma região paralela.

Identificador de Thread

- Cada *thread* em execução possui um número único que a distingue de todas as demais.
- A função utilizada para encontrar o número atual da thread em execução é `omp_get_thread_num(void)`:

C/C++:

```
#include <omp.h>
```

```
int omp_get_thread_num(void);
```

- Esta função retorna valores entre 0 e '`omp_get_num_threads()` - 1'

Exemplo

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        int nthreads = omp_get_num_threads();

        printf("Thread %d de %d\n", id, nthreads);
    }
    return 0;
}
```

Identificador de Thread

- Uma saída possível para a execução do programa, após sua compilação, seria a seguinte:

Thread 0 de 4

Thread 1 de 4

Thread 2 de 4

Thread 3 de 4

Identificador de Região Paralela

- A função `omp_in_parallel()` é utilizada para identificar se a execução atual é dentro de uma região paralela. Sua sintaxe é:

C/C++:

```
#include <omp.h>
```

```
int omp_in_parallel();
```

- Um valor diferente de zero (considerado *true* em um contexto booleano) é retornado se a thread chamadora estiver dentro de uma região paralela.

Número de processadores

- A função `omp_get_num_procs()` retorna o número de processadores/núcleos disponíveis no sistema para a execução do programa no momento da chamada.

C/C++:

```
#include <omp.h>
```

```
int omp_get_num_procs(void);
```

- Pode incluir no total o número de processadores físicos e lógicos. É um valor geralmente estático para a execução, refletindo a capacidade de paralelismo da máquina.

Exemplo

```
#include <stdio.h>
#include <omp.h>
int main() {
    if (!omp_in_parallel()) printf ("Fora da região paralela \n");
    int numt = omp_get_num_procs();
    #pragma omp parallel num_threads(numt)
    {
        int id = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
        printf("Thread %d de %d\n", id, nthreads);
    }
    return 0;
}
```

Exemplo

- Um resultado possível da execução deste código seria:

Fora da região paralela

Thread 2 de 4

Thread 3 de 4

Thread 1 de 4

Thread 0 de 4

Diretiva single

- A diretiva **single** define que um bloco estruturado, dentro de uma região paralela, deve ser executado por uma única *thread*.
- A primeira *thread* que alcançar a diretiva **single** irá executar o bloco, as demais devem esperar o final da execução do bloco, ou seja, existe uma **barreira implícita** no fim do bloco estruturado.

```
#pragma omp single [cláusulas]
{
    bloco estruturado
}
```

Diretiva master

- Um bloco estruturado com a diretiva **master** deve ser executado apenas pela *thread master* (tid = 0). As demais *threads* pulam o bloco estruturado e continuam a sua execução.
- Nesse aspecto, seu comportamento é diferente da diretiva **single**, ou seja, não existe uma barreira implícita ao final da região paralela. Na maior parte das vezes é utilizada para operações de E/S.

```
#pragma omp master
{
    bloco estruturado
}
```


Exemplo

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel num_threads(4)
    {
        int tid = omp_get_thread_num();
        printf("Thread %d entrou na região paralela\n", tid);
        #pragma omp single
            printf("Thread %d na região single\n", tid);

        #pragma omp master
            printf("Thread %d na região master\n", tid);
    }
    return 0;
}
```

Exemplo

- Um possível resultado da execução deste exemplo seria:

Thread 3 entrou na região paralela
Thread 3 na região single
Thread 0 entrou na região paralela
Thread 2 entrou na região paralela
Thread 1 entrou na região paralela
Thread 0 na região master

Roteiro

- Medindo o tempo de execução
- Diretiva **for**
- Diretiva **parallel for**
- Escopo das variáveis.
- Cláusulas **shared, private, default, firstprivate, lastprivate**
- Cláusula **if**
- Cláusula **reduction**
- Quais os tipos de escalonamento disponíveis para a diretiva **for**?



Medindo o tempo de execução

- A função `omp_get_wtime()` é utilizada no OpenMP para retornar o tempo decorrido, em segundos, a partir de um mesmo ponto do início da execução do programa. Sua sintaxe é:

C/C++:

```
#include <omp.h>
```

```
double omp_get_wtime (void);
```

- Assim, basta realizarmos a chamada uma vez no início e outra vez no final do programa, que a diferença entre os valores vai informar o tempo em segundos gasto para execução do trecho do programa em questão.



Medindo o tempo de execução

```
#include <omp.h>
#include <stdio.h>
int main (int argc, char *argv[ ]) { /* omp_numprocs.c */
    double inicio, fim;
    inicio = omp_get_wtime();
    if (!omp_in_parallel()) /* Verifica se está na região paralela */
        printf("Fora da região paralela \n");
    #pragma omp parallel
    {
        int num_procs, max_threads, tid;
        tid = omp_get_thread_num();
        if (tid == 0) { /* Apenas as thread master faz isto */
            num_procs = omp_get_num_procs();
            printf("Número de processadores disponíveis = %d\n", num_procs);
            if (omp_in_parallel()) /* Verifica se está na região paralela */
                printf("Dentro da região paralela \n");
        }
    } /* Todas as threads se juntam à thread master e terminam */
    fim = omp_get_wtime();
    printf("Tempo gasto na execução = %3.6f segundos. \n", fim-inicio);
    printf("Precisão da medida = %3.9f segundos\n", omp_get_wtick());
    return(0);
}
```



Medindo o tempo de execução

- Um possível resultado da execução deste programa seria:

```
gabriel@UFRJ:~/OPENMP$ ./bin/omp_numprocs  
Fora da região paralela  
Número de processadores disponíveis = 12  
Número máximo de threads = 12  
Dentro da região paralela  
Tempo gasto na execução = 0.010408 segundos.  
Precisão da medida = 0.0000000001 segundos
```



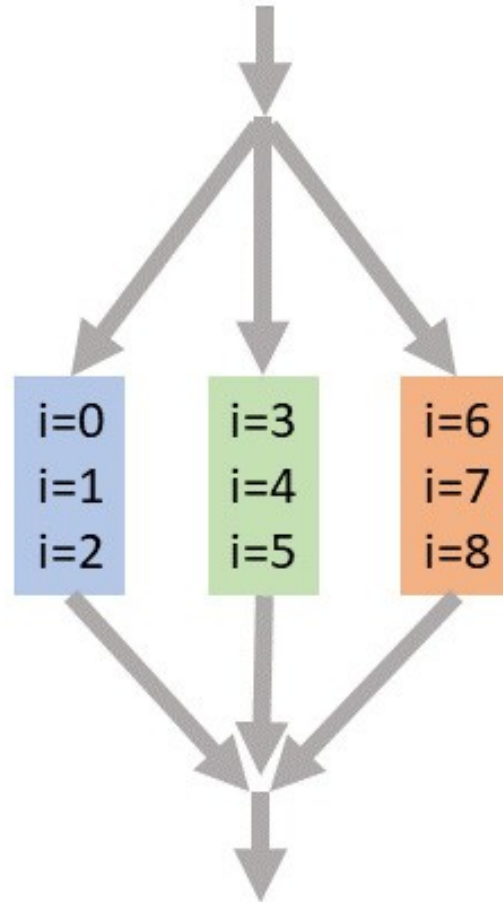
Diretiva for

- Laços são a maior fonte de paralelismo na maioria dos códigos.
- A diretiva **for** permite a extração de paralelismo dos laços com a divisão da execução das iterações do laço entre as diversas *threads* de um time.
- Apresentaremos inicialmente apenas a forma básica dessa diretiva.

```
#pragma omp for [cláusulas]  
laço for
```



Diretiva for



```
...  
  
#pragma omp parallel  
{  
  
#pragma omp for  
for (int i = 0; i < 9 ; ++i) {  
    a[i] = i;  
}  
  
}  
  
...
```



Diretiva for

- Sem cláusulas adicionais, a diretiva **for** deve particionar as iterações do laço o mais igualmente possível entre as threads.
- Contudo, isso é dependente de implementação e ainda há alguma ambiguidade, por exemplo, havendo 7 iterações para serem divididas entre 3 threads, elas poderão ser particionadas como $3+3+1$ ou $3+2+2$.
- A variável de índice do laço tem o seu valor indeterminado ao final do laço, a menos que uma cláusula **lastprivate**, a ser vista mais adiante em detalhes, seja especificada.



Diretiva for

- A diretiva **for** possui as seguintes restrições:
 - Deve ser invocada dentro de uma região paralela.
 - O corpo do laço deve ser um bloco estruturado, e sua execução não pode ser terminada por um comando **break**.
 - Os valores para o controle das expressões do laço associados com a diretiva **for** devem ser os mesmos para todas as *threads* do laço.
 - A variável de índice do laço deve ser do tipo inteiro com sinal.
- Ao final do laço existe uma **barreira implícita**, ou seja, todas as *threads* participantes da computação do laço devem aguardar pelas demais, para só então prosseguir.



Diretiva parallel for

- A construção for é tão comum que existe uma forma combinada da diretiva **parallel** com a diretiva **for**:

```
#pragma omp parallel for [cláusulas]  
    laço for
```



Diretiva parallel for

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) { /* omp_for.c */
    #pragma omp parallel for num_threads(4)
    for (int i = 0; i < 18; ++i) {
        printf("Iteração %2d executada pela thread %d \n", i, omp_get_thread_num());
    }
    return(0);
}
```



Diretiva parallel for

```
gabriel@UFRJ:~/OPENMP$ ./bin/omp_for
Iteração 10 executada pela thread 2
Iteração 11 executada pela thread 2
Iteração 12 executada pela thread 2
Iteração 13 executada pela thread 2
Iteração 14 executada pela thread 3
Iteração 15 executada pela thread 3
Iteração 16 executada pela thread 3
Iteração 17 executada pela thread 3
Iteração 5 executada pela thread 1
Iteração 6 executada pela thread 1
Iteração 7 executada pela thread 1
Iteração 8 executada pela thread 1
Iteração 9 executada pela thread 1
Iteração 0 executada pela thread 0
Iteração 1 executada pela thread 0
Iteração 2 executada pela thread 0
Iteração 3 executada pela thread 0
Iteração 4 executada pela thread 0
gabriel@UFRJ:~/OPENMP$ |
```



Cláusula if

Sintaxe:

if (expressao)

- Onde, **expressao** é uma expressão inteira que, se avaliada como verdadeira (não nula), faz com que o código na região paralela seja executado em paralelo.
- Se a expressão for avaliada como **falsa** (zero) a região paralela é executada sequencialmente (por uma única *thread*).



Cláusula if

```
double dot (const size_t n, double *x, double *y) {  
    double prodint = 0;  
    #pragma omp parallel for reduction(+:prodint) if(n>=1000)  
    for (size_t i = 0; i < n; ++i )  
        prodint += x[i] * y[i];  
    return f;  
}
```

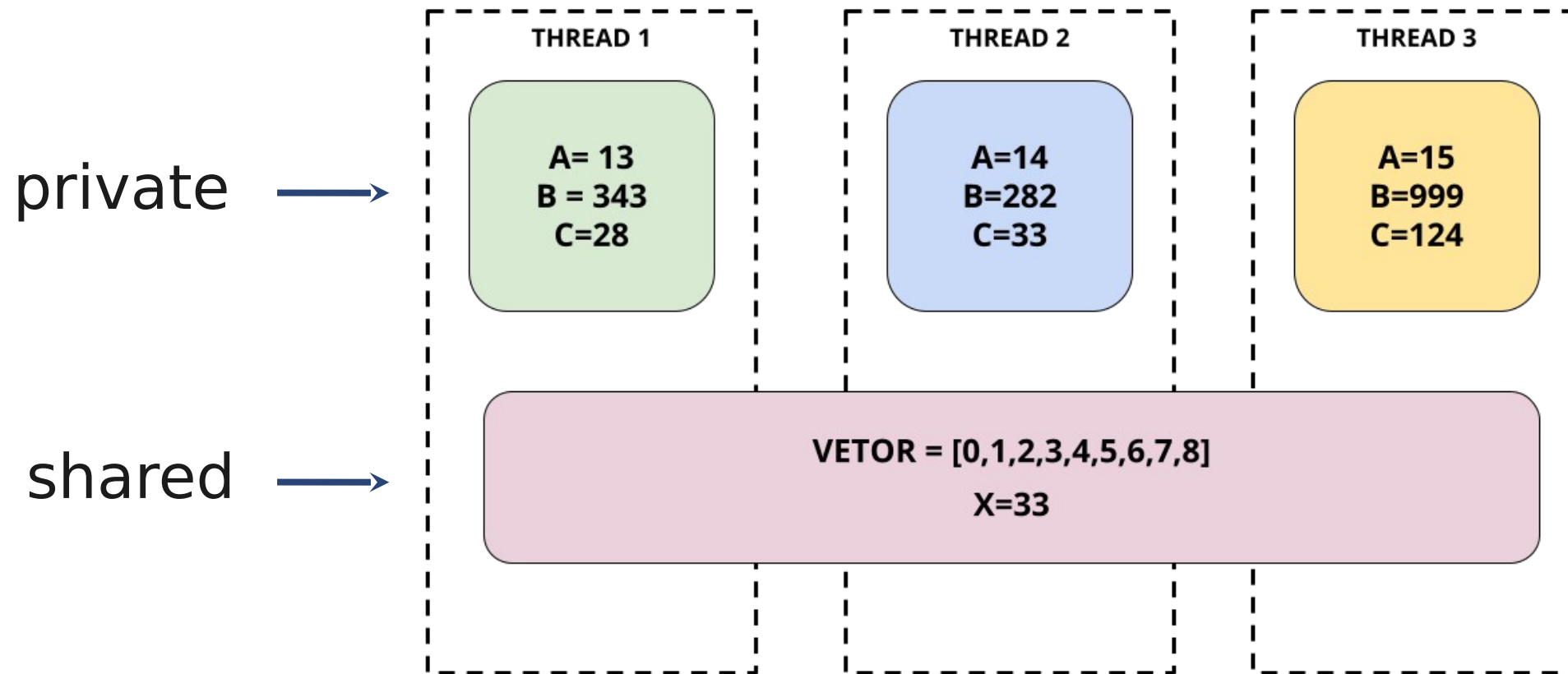


Escopo das variáveis

- Dentro de uma região paralela, as variáveis podem ser **privadas** ou **compartilhadas**.
- As variáveis **compartilhadas**, cujo conteúdo é o mesmo em todas as *threads*, podem ser lidas ou escritas a partir de **qualquer thread**.
- Por outro lado, cada *thread* tem a sua própria cópia de variáveis privadas.
- Essas variáveis, embora possam ter o mesmo nome em um programa, são invisíveis para as outras *threads*.
- Uma variável **privada** pode ser lida ou escrita apenas pela sua **própria thread**.



Escopo das variáveis



Escopo das variáveis

- As variáveis declaradas **fora** da região paralela são **compartilhadas** por padrão.
- As variáveis declaradas **dentro** da região paralela são **privadas**.
- Mas se as variáveis forem declaradas dentro da região paralela com **static** serão **compartilhadas** por padrão.
- As variáveis de **iteração do laço mais externo** dentro da região paralela são **privadas**.
- **As demais variáveis de laços internos são compartilhadas por padrão.**
- O comportamento padrão dentro de uma região paralela pode ser alterado por cláusulas como **default**, **shared**, **private**, **firstprivate** e **lastprivate**.



Cláusulas **shared**, **private** e **default**

- A sintaxe para essas três cláusulas é bem simples, sendo que elas devem sempre acompanhar uma diretiva **parallel** ou **parallel for**.
 - **shared(lista-de-variáveis)**
 - **private(lista-de-variáveis)**
 - **default(shared | none)**
- A cláusula **shared** declara como compartilhadas as variáveis listadas em **lista-de-variáveis**, assim como a cláusula **private** as declara como privadas.
- A ausência de uma cláusula explícita na diretiva **parallel** tem o comportamento padrão de uma cláusula **default(shared)**.



Cláusulas default

- A cláusula **default** tem dois parâmetros possíveis: **shared** e **none**.
- Ao se especificar **default(shared)**, as variáveis não listadas serão **compartilhadas** por padrão.
- Contudo, ao se especificar **default(none)**, a situação é um pouco mais complicada:
 - Se a variável não foi listada na cláusula **shared**; não foi declarada dentro da região paralela; não for uma constante; ou não for o índice mais externo de um laço com uma diretiva **for** ou **parallel for**; o compilador irá emitir uma **mensagem de erro**, indicando que a variável não foi especificada como **shared** ou **private**.



Cláusulas shared, private e default

- Como decidir se uma variável deve ser compartilhada ou privada?
 - A maioria das variáveis são compartilhadas.
 - Os índices dos laços são privados.
 - As variáveis temporárias dos laços são compartilhadas.
 - As variáveis apenas de leitura são compartilhadas.
 - As matrizes principais são compartilhadas.
 - As variáveis declaradas localmente à região paralela são privadas.
- Quando uma variável compartilhada pode ser modificada por várias *threads* ao mesmo tempo, devem-se usar mecanismos de sincronização para evitar inconsistências.



Cláusula shared

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main(int argc, char *argv[]) { /* omp_shared.c */
    int i, n = 7;
    int a[n];
    (void) omp_set_num_threads(4);
    for (i = 0; i < n; i++)
        a[i] = i+1;
    #pragma omp parallel for shared(a)
    for (i = 0; i < n; i++) {
        a[i] += i;
    } /*-- Fim do parallel for --*/
    printf("No programa principal depois do parallel for:\n");
    for (i = 0; i < n; i++)
        printf("a[%d] = %d\n", i, a[i]);
    return(0);
}
```



Cláusula shared

- Como o vetor **a** é compartilhado entre todas as *threads*, a saída desse programa, que é sempre executado com quatro threads, será a seguinte:

```
No programa principal depois do parallel for:  
a[0] = 1  
a[1] = 3  
a[2] = 5  
a[3] = 7  
a[4] = 9  
a[5] = 11  
a[6] = 13
```



Cláusula firstprivate

- As variáveis privadas não tem valor inicial no início da região paralela.
- Algumas vezes é necessário que uma variável local tenha um valor inicial dentro da região paralela. Sintaxe:
- C/C++:

firstprivate(lista-de-variaveis)



Cláusula firstprivate

```
int base = 10;  
#pragma omp parallel for firstprivate(base)  
for (int i = 0; i < N; i++) {  
    base += i;    // começa com 10 em cada thread  
}
```



Cláusula lastprivate

- Algumas vezes é necessário que se saiba o valor que uma variável privada terá na saída de um laço **for** ou **parallel for** (normalmente é indefinido).

Sintaxe:

- C/C++:

lastprivate(lista-de-variaveis)



Cláusula lastprivate

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main(int argc, char *argv[]) { /* omp_lastprivate.c */
    int i;
    int x = 33;
    #pragma omp parallel for lastprivate(x)
    for (i = 0; i <= 10; i++) {
        x = i;
        printf("Número da thread: %d x: %d\n", omp_get_thread_num(), x);
    }
    printf("x é %d\n", x);
    return(0);
}
```



Cláusula lastprivate

```
Número da thread: 0    x: 0  
Número da thread: 0    x: 1  
Número da thread: 0    x: 2  
Número da thread: 3    x: 9  
Número da thread: 3    x: 10  
Número da thread: 2    x: 6  
Número da thread: 2    x: 7  
Número da thread: 2    x: 8  
Número da thread: 1    x: 3  
Número da thread: 1    x: 4  
Número da thread: 1    x: 5  
x é 10
```



Cláusula reduction

- Uma redução produz um único valor de uma variável, cujo valor é local nas diversas *threads*, a partir de operações tais como soma, multiplicação, máximo, mínimo, “e” e “ou”. Vejamos o exemplo apresentado seguir:

```
b = 0;
```

```
    for (i=0; i<n; i++)
```

```
        b += a[i];
```

- Se a variável “b” fosse compartilhada por todas as *threads*, e permitindo-se o acesso de apenas uma *thread* por vez, removeria todo o paralelismo.
- Ao invés disso, cada *thread* pode acumular uma soma parcial em sua própria cópia privada, e então essas cópias são reduzidas para dar o resultado final.



Cláusula reduction

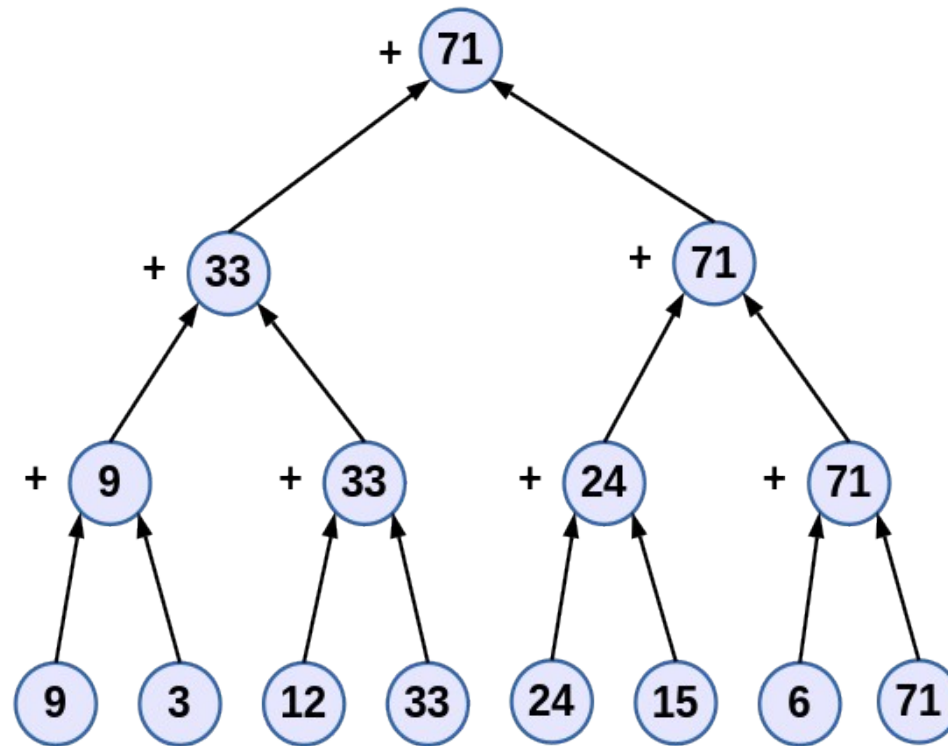
- Uma redução produz um único valor a partir de operações associativas como adição, multiplicação, máximo, mínimo, e, ou.
- É desejável que cada *thread* faça a redução em uma cópia privada e então reduzam todas elas para obter o resultado final.
- Uso da cláusula **reduction**:

C/C++:

reduction(op:list)



Operação de Redução (Máximo)



Cláusula reduction

Operador	Operação	Valor Inicial
+	soma	0
-	subtração	0
*	multiplicação	1
&	“e” bit a bit	todos os bits em 1
	ou bit a bit	0
^	“ou exclusivo” bit a bit	0
&&	“e” lógico	1
	“ou” lógico	0
max	máximo	menor repres.
min	mínimo	maior repres.



Cláusula reduction

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#define SOMA_INICIAL 1000
int main(int argc, char *argv[]) {    /* omp_reduction.c */
    int i, n = 25;
    int soma, a[n];
    int ref = SOMA_INICIAL + ((n-1)*n)/2;
    omp_set_num_threads(4);
    for (i = 0; i < n; i++)
        a[i] = i;
    soma = SOMA_INICIAL;
    printf("Valor da soma antes da região paralela: %d \n", soma);
    #pragma omp parallel default(none) shared(n,a) reduction(+:soma)
    {
        soma=0;
        #pragma omp for
        for (i = 0; i < n; i++)
            soma += a[i];
        printf("Valor da soma na thread: %d é igual %d\n", omp_get_thread_num(), soma);
    } /*-- Fim da redução paralela --*/
    printf("Valor da soma depois da região paralela: %d\n", soma);
    printf("Verificação do resultado: soma = %d (deveria ser %d)\n", soma, ref);
    return(0);
}
```



Cláusula reduction

```
Valor da soma antes da região paralela: 1000  
Valor da soma na thread: 3 é igual 129  
Valor da soma na thread: 0 é igual 21  
Valor da soma na thread: 2 é igual 93  
Valor da soma na thread: 1 é igual 57  
Valor da soma depois da região paralela: 1300  
Verificação do resultado: soma = 1300 (deveria ser 1300)
```



Cláusula reduction

- A partir da versão 4.5 do OpenMP, os arranjos também podem ser usados como variáveis de redução (anteriormente isso só era possível com escalares e elementos de um arranjo), como apresentado a seguir.

```
#pragma omp parallel for private (i) reduction (+:b)
for (j = 0; j < N; j++)
    for (i = 0; i < M; i++)
        b(i) = b(i) + b(i,j)
```



Cláusulas da diretiva for

- A diretiva **for** admite também as seguintes cláusulas:
 - **schedule**
 - **ordered**
 - **collapse**
- Além disso, a diretiva **parallel for** admite também todas as cláusulas da diretiva **parallel**.



Cláusula `schedule`

- Vamos continuar o estudo das principais cláusulas da diretiva **for**, tais como:
 - **schedule**
- A cláusula **schedule** permite uma variedade de opções para especificar quais *threads* executarão as iterações dos laços.
- Sintaxe:
schedule (tipo[, tamanho])
- Onde *tipo* pode ser **static**, **dynamic**, **guided**, **auto** ou **runtime** e *tamanho* é uma expressão inteira com valor positivo.



Escalonamento static

- No escalonamento **static**, sem a especificação de tamanho, o espaço de iteração é dividido em pedaços (aproximadamente) iguais e cada pedaço é atribuído a cada *thread*, no que é chamado de escalonamento em bloco.
- Se o valor de tamanho for especificado, o espaço de iteração é dividido em pedaços, cada um com **tamanho** iterações, e os pedaços são atribuídos ciclicamente a cada *thread*, o que é chamado de escalonamento cíclico por bloco.



Escalonamento dynamic

- O escalonamento **dynamic** divide o espaço de iteração em pedaços com **tamanho** iterações, e os atribui para as *threads* com uma política “primeiro a chegar, primeiro a ser atendido”.
- Isto é, se uma *thread* terminou um pedaço, ela receberá o próximo pedaço da lista.
- Quando nenhum valor de **tamanho** for especificado, o valor padrão assumido é '1'.



Escalonamento guided

- O escalonamento **guided** é similar ao **dynamic**, mas os pedaços iniciam grandes e se tornam menores exponencialmente.
- O tamanho do próximo pedaço é (com arredondamento para cima) o número de iterações restantes dividido pelo número de threads.
- O valor tamanho especifica o tamanho mínimo dos pedaços, exceto para o pedaço com as últimas iterações, que pode ser menor.
- Quando nenhum valor de tamanho for especificado, o padrão é igual a



Escalonamento auto e runtime

- O escalonamento **auto** dá liberdade para o ambiente de execução escolher qual método de escalonamento será utilizado.
- Isso pode ser útil, por exemplo, quando o laço paralelo é executado diversas vezes, permitindo ao ambiente de execução convergir para um tipo de escalonamento com bom balanceamento de carga e baixa sobrecarga.
- O escalonamento **runtime** delega a escolha do escalonamento para o momento da execução do programa, podendo ser determinado pelo valor da variável de ambiente OMP_SCHEDULE ou pela chamada à função `omp_set_schedule()`.

```
$ export OMP_SCHEDULE=tipo[,tamanho]
```



Escalonamento laços for

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) { /* omp_schedule.c */
    int n = 20;

    printf("Clausula static SEM o parametro tamanho \n");
    #pragma omp parallel for schedule (static) num_threads(4)
    for (int i = 0; i < n; ++i) {
        printf("tid =%d iteracao = %d \n", omp_get_thread_num(), i);
    }

    printf("Clausula static COM o parametro tamanho \n");
    #pragma omp parallel for schedule (static,3) num_threads(4)
    for (int i = 0; i < n; ++i) {
        printf("tid =%d iteracao = %d \n", omp_get_thread_num(), i);
    }

    return 0;
}
```



Escalonamento laços for

- A saída deste programa mostra a divisão das iterações com o uso do escalonamento **static**, sem e com um parâmetro **tamanho**.
- No primeiro caso as iterações são divididas o mais igualmente possível entre as *threads*.
- No segundo caso, a cada conjunto de 3 iterações, a execução das *threads* é alternada, mantendo-se sempre a mesma ordem.
- O programa pode ser executado variando-se o tipo de escalonamento.



Escalonamento laços for

- Qual tipo de escalonamento deve ser utilizado
 - **static**: melhor para laços balanceados – menor sobrecarga.
 - **static,n**: melhor para laços com desbalanceamento suave.
 - **dynamic**: útil se as iterações tem grande variação de carga, mas acaba com a localidade espacial dos dados e tem o custo adicional de troca de *threads*.
 - **guided**: frequentemente menos cara que dynamic, mas tenha cuidado com laços onde as primeiras iterações são as mais demoradas.
 - **auto**: pode ser útil se o laço é executado diversas vezes ao longo da execução do programa com dados diferentes.
 - **runtime**: use para uma experimentação adequada.



Roteiro

- Vamos continuar o estudo de algumas cláusulas de modificação do comportamento de laços, tais como: **ordered**, **collapse**
- E também vamos ver algumas diretivas de sincronização, tais como: **barrier**, **nowait**
- Quais as diretivas e funções de sincronização do OpenMP?
- Como empregar as seguintes diretivas: **critical**, **atomic**
- Quais são as funções de *lock* disponíveis no OpenMP?
- Regras para utilização das diretivas e funções de sincronização?



Diretiva ordered

- A diretiva **ordered** pode ser utilizada dentro de uma região for ou parallel for com uma cláusula ordered especificada.
- Essa diretiva serve para indicar que um trecho de código dentro do laço deverá ser executado na mesma ordem na qual seria executado sequencialmente.
- A diretiva **ordered** não pode ser empregada sem a cláusula **ordered** correspondente, ou seja, não se pode utilizar uma diretiva **ordered** dentro de um laço com uma diretiva for que não tenha uma cláusula **ordered**.
- Sintaxe:

```
#pragma omp ordered
```



Diretiva ordered

Exemplo:

```
#pragma omp parallel for ordered
    for (j =0; j < n; j++)
        . . .
#pragma omp ordered
    printf ("%d %d \n", j,count[j])
    . . .
```



Diretiva ordered

```
#include <stdio.h>
#include <omp.h>
void teste(int first, int last) {
    #pragma omp parallel for schedule(static) ordered
        for (int i = first; i <= last; ++i) {
            // Faz alguma coisa aqui
            if (i % 2) {
                #pragma omp ordered
                printf("teste() iteração %d\n", i);
            }
        }
    }
```



Cláusula collapse

- A cláusula collapse é utilizada para aumentar o número total de iterações que serão particionadas entre as *threads* disponíveis para execução, aumentando assim a granularidade do trabalho a ser realizado por cada thread.
- Se a quantidade de trabalho a ser realizado por cada *thread* for significativa (após a aplicação da compactação), isso poderá melhorar a escalabilidade paralela programa.
- Vejamos a seguir um exemplo do seu uso:



Cláusula collapse

```
#include <stdio.h>
int main(int argc, char *argv[]) { /* omp_collapse.c */
int j, k, jlast, klast;
#pragma omp parallel
{
    #pragma omp for collapse(2) lastprivate(jlast, klast)
    for (k = 1; k <= 2; k++)
        for (j = 1; j <= 3; j++) {
            jlast = j;
            klast = k;
        }
    #pragma omp single
    printf("%d %d\n", klast, jlast);
}
}
```



Cláusula collapse

```
#pragma omp parallel for collapse(2)
/* Uso não otimizado */
    for (i = 0; i < imax; i++) {
        for (j = 0; j < jmax; j++)
            a[j + jmax*i] = 1.;
    }
```

```
#pragma omp parallel for collapse(2)
/* Uso recomendado */
    for (i = 0; i < imax; i++) {
        for (j = 0; j < jmax; j++)
            a[k++] = 1.;
    }
```



Sincronização

- A sincronização é necessária para ordenar o acesso às variáveis compartilhadas, sendo utilizada para assegurar a ordem correta de leituras e escritas, protegendo a atualização das variáveis compartilhadas (que não são atômicas por padrão).
- Suponhamos, por exemplo, duas *threads* tentando incrementar a mesma variável X.
- A *thread* 1 deve escrever na variável X antes que *thread* 2 faça a sua leitura, ou a *thread* 1 deve ler a variável X depois que a *thread* 2 fizer sua escrita.



Sincronização

- É importante observar que as operações para atualização de variáveis compartilhadas (p.ex. $X = X + 1$) não são atômicas!
- Se duas *threads* tentarem fazer isto ao mesmo tempo, uma das atualizações pode ser perdida e resultar em um valor final da variável **incorreto**.



Sincronização

- O que é necessário?
 - É necessário sincronizar ações em variáveis compartilhadas.
 - É necessário assegurar a ordenação correta de leituras e escritas.
 - É necessário proteger a atualização de variáveis compartilhadas (não atômicas por padrão).
- O OpenMP dispõe de diversas diretivas e funções para assegurar a correta sincronização entre as diversas *threads* em execução. Entre elas, destacamos:
 - Diretiva **barrier**
 - Cláusula **nowait**



Diretiva barrier

- Enquanto todas as *threads* não chegarem a uma barreira, nenhuma delas pode prosseguir além desse ponto.
- Com isso, pode-se garantir que certas fases da computação terminaram, antes de avançar na execução do programa.
- Devemos notar que há uma barreira implícita no final das diretivas **for**, **sections** e **single**.
- Adicionalmente, no final de uma região paralela o time de threads é dissolvido e apenas a thread master continua, ou seja, há uma barreira implícita no final da região paralela.



Diretiva barrier

```
#include <omp.h>
#include <stdio.h>
int main(int argc, char *argv[])    /*omp_barrier.c */
{
    #pragma omp parallel num_threads(4)
    {
        int tid = omp_get_thread_num();
        int num_threads = omp_get_num_threads();
        if (tid == 0 ) {
            printf ("Estou atrasada para a barreira! Tecle enter\n");
            getchar();
        }
        #pragma omp barrier
        printf("Passei da barreira. Eu sou a %d de %d threads \n", tid,
            num_threads);
    }
    return 0;
}
```



Cláusula **nowait**

- A cláusula **nowait** pode suprimir as barreiras implícitas no final das diretivas **for**, **sections** e **single**.
- O objetivo é aumentar o paralelismo possível em um programa, pois as barreiras podem ser muito caras em termos de redução desse paralelismo.
- Mas deve-se ter cuidado para não tirar barreiras que estejam garantindo a execução correta do programa, como entre dois laços com dependências de dados.
- Note que, por motivos quase óbvios, não é possível utilizar a cláusula **nowait** com as diretivas **parallel for** e **parallel sections**, já que no final de toda região paralela existe uma barreira implícita, que não pode ser removida.



Cláusula nowait

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (j = 0; j < n; j++) {
        a[j] = c * b[j];
    }
    #pragma omp for nowait
    for (i = 0; i < m; i++) {
        x[i] = sqrt(y[i]) * 2.0;
    }
}
```



Diretiva critical

- Uma seção crítica é um bloco de código executado apenas por uma *thread* de cada vez, sendo normalmente utilizado para proteger a atualização de variáveis compartilhadas.
- As seções críticas de uma diretiva **critical** podem receber nomes.
- Se uma *thread* está em uma seção crítica com um dado nome, nenhuma outra *thread* poderá estar em uma seção crítica com o mesmo nome (embora possa haver mais de uma *thread* em seções críticas com nomes diferentes).



Diretiva critical

- Sintaxe:

```
#pragma omp critical [( nome )]  
    bloco estruturado
```

- Se o nome foi omitido, um nome nulo é assumido (todas as seções críticas sem nome tem efetivamente o mesmo nome).



Diretiva critical

```
#include <omp.h>
#include <stdio.h>
int main(int argc, char *argv[]) { /* omp_critical.c */
    int conta_secao = 0;
    #pragma omp parallel num_threads(4)
    {
        #pragma omp single nowait
        {
            int tid = omp_get_thread_num();
            #pragma omp critical
            conta_secao++;
            /* deve imprimir o número um ou dois */
            printf( "Ordem de execução da sessão %d tid= %d \n", conta_secao, tid);
        }
        #pragma omp single nowait
        {
            int tid = omp_get_thread_num();
            #pragma omp critical
            conta_secao++;
            /* deve imprimir o número um ou dois */
            printf( "Ordem de execução da sessão %d tid= %d \n", conta_secao, tid);
        }
    }
    return(0);
}
```



Diretiva critical

- Um possível resultado da execução seria:

Ordem de execução da sessão 2 tid= 1

Ordem de execução da sessão 1 tid= 2

- Ou seja, garante-se que o incremento da variável será feito de maneira adequada, resultando sempre na impressão dos valores 1 e 2, embora a ordem exata não possa ser garantida.
- Se não houvesse a diretiva critical, eventualmente as duas *threads* poderiam imprimir mesmo valor, no caso 1.



Diretiva **atomic**

- A diretiva **atomic** permite acessar uma variável de modo atômico.
- As condições de corrida são evitadas através do controle direto das *threads* concorrentes que podem ler ou escrever em um mesma variável.
- Com o uso da diretiva **atomic**, programas paralelos mais eficientes podem ser escritos e com um número menor de travas,
- Objetos que possam ser atualizados em paralelo e estejam sujeitos a condições de corrida devem ser protegidos com a diretiva **atomic**.



Diretiva atomic

- Sintaxe:

`#pragma omp atomic [cláusulas]`

sentença ou bloco estruturado

- Onde *sentença* é uma expressão com um tipo escalar.
- O *bloco estruturado* pode envolver até duas sentenças do tipo anterior.



Cláusulas da diretiva atomic

- **update:** Diz ao compilador que a memória será lida, modificada por uma expressão matemática simples (como ++, --, +=, *=) e escrita de volta atômicamente. Os processadores modernos possuem instruções de máquina (como LOCK XADD na arquitetura x86) que fazem exatamente isso em um único ciclo de relógio, sem travar as outras threads que estão atualizando variáveis diferentes. É a operação padrão para a diretiva.
- **read:** Utilizada apenas para ler um valor de forma atômica, garantindo que você não pegue uma escrita pela metade sendo efetuada por outra thread.
- **write:** Utilizada apenas quando se escreve um valor diretamente na memória de forma atômica (ex: `x = 10;`).
- **capture:** É usada quando você precisa atualizar o valor e ao mesmo tempo salvar o resultado (ou o valor antigo) em uma variável local para usar logo em seguida (ex: `v_local = x++;`). É uma diferença sutil mas importante, já que essa operação não pode ser realizada por uma única instrução do processador.



Exemplo de update, read e write

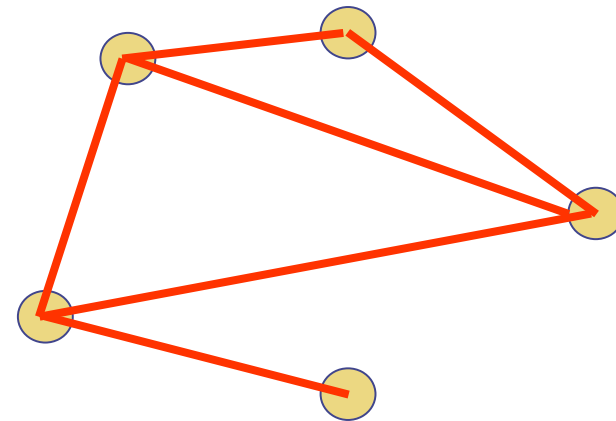
```
extern int x[10];  
extern int f(int);  
int temp[10], i;  
  
for (i = 0; i < 10; i++) {  
    #pragma omp atomic read  
    temp[i] = x[f(i)];  
  
    #pragma omp atomic write  
    x[i] = temp[i]*2;  
  
    #pragma omp atomic update  
    x[i] *= 2;  
}
```



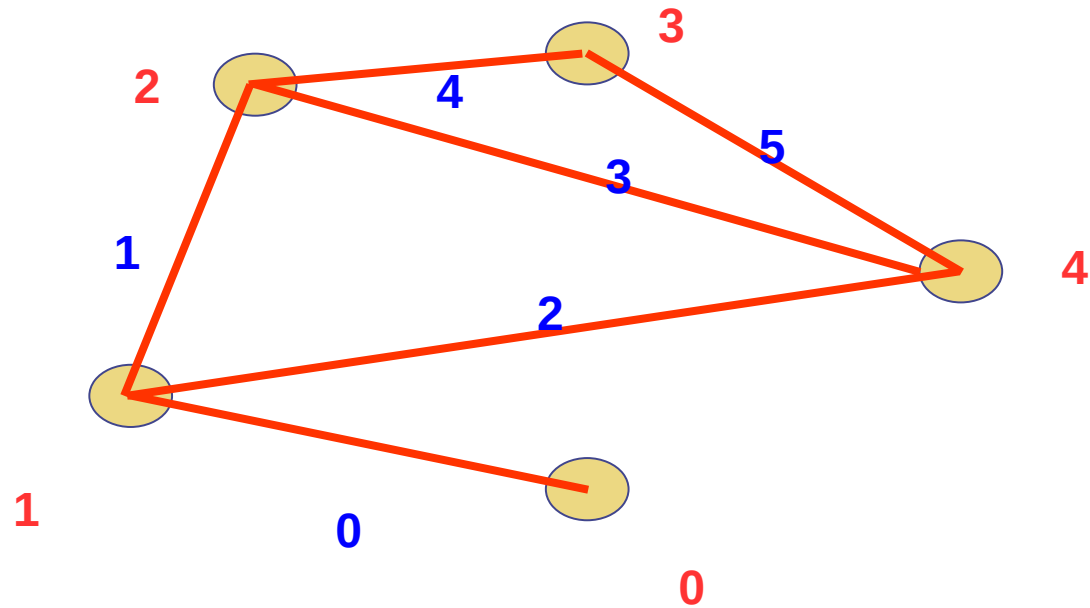
Diretiva atomic

- Exemplo (computar o grau de cada vértice em um grafo):

```
#pragma omp parallel for
    for (j=0; j<nedges; j++){
        #pragma omp atomic
        degree[edge[j].vertex1]++;
        #pragma omp atomic
        degree[edge[j].vertex2]++;
    }
```



Diretiva atomic



Funções de trava (lock)

- Ocasionalmente pode ser necessário mais flexibilidade que a fornecida pelas diretivas `critical` e `atomic`.
- Um *lock* é uma variável especial que pode ser marcada por uma *thread*. Nenhuma outra thread pode marcar o *lock* até que a *thread* que o marcou o desmarque.
- Marcar um *lock* pode tanto pode ser bloqueante como não bloqueante.
- Um *lock* deve ter um valor inicial antes de ser usado e pode ser destruído quando não for mais necessário.
- Variáveis de *lock* não devem ser usadas para qualquer outro propósito.



Funções de trava (lock)

- Sintaxe:
- C/C++:

```
#include <omp.h>
void omp_init_lock(omp_lock_t *lock);
void omp_set_lock(omp_lock_t *lock);
int omp_test_lock(omp_lock_t *lock);
void omp_unset_lock(omp_lock_t *lock);
void omp_destroy_lock(omp_lock_t *lock);
```



Funções de trava (lock)

```
omp_init_lock(&lock)
#pragma omp parallel shared(&lock)
...
do {
    do_something_else(); }
while (~ omp_test_lock(&lock))

work();
omp_unset_lock(&lock);
...
```



Regras para sincronização

- A diretiva **barrier** é a mais utilizada para sincronização do trabalho das *threads* e as barreiras implícitas só devem ser removidas com uso da cláusula **nowait** se houver certeza que isso não afeta o funcionamento do código.
- Como uma regra simples, use a diretiva **atomic** sempre que possível, já que permite o máximo de otimização.
- Se não for possível use a diretiva **critical**. Tenha cuidado de usar diferentes nomes sempre que possível.
- Como um último recurso você pode ter que usar as rotinas de **lock**, mas isto deve ser uma ocorrência muito rara.



Variáveis de Ambiente

- **OMP_STACKSIZE**

- Caso grandes estruturas de dados privadas sejam utilizadas, é possível que o programa estoure o limite de espaço de pilha definido para sua utilização.
- Nesse caso, o tamanho das pilhas do time de *threads*, além da thread master, pode ser controlado pela variável de ambiente OMP_STACKSIZE.
- O tamanho da pilha específico da *thread* master é definido como em um programa sequencial, isto é, com o uso das opções do compilador ou com o comando ulimit.



Variáveis de Ambiente

- O valor deve ser um inteiro n concatenado com B, K, M, e G para especificar, respectivamente, bytes, kilobytes (1024 bytes), megabytes (1024 kilobytes), ou gigabytes (1024 megabytes), para especificar o tamanho da pilha:

```
$ setenv OMP_STACKSIZE "3000 k"
```

```
$ setenv OMP_STACKSIZE 10M
```

```
$ setenv OMP_STACKSIZE "20 M"
```

```
$ setenv OMP_STACKSIZE "1G"
```

```
$ setenv OMP_STACKSIZE 20000
```



Variáveis de Ambiente

- **OMP_DYNAMIC**

- Se o valor dessa variável de ambiente for definido como **true**, o ambiente de execução pode ajustar o número de *threads* para a execução de regiões paralelas de modo a otimizar o uso de recursos do sistema.
- Se o valor da variável for definido como **false**, isso impede o ambiente de execução entregue um número de *threads* menor que o solicitado.
- O valor padrão desta variável de ambiente é dependente de implementação, assim como o comportamento do programa, se nenhum valor for definido.

```
$ setenv OMP_DYNAMIC true
```



Variáveis de Ambiente

- **OMP_PLACES**

- A variável de ambiente **OMP_PLACES** especifica uma lista de lugares para a execução das threads no OpenMP.
- Essa localização das threads pode ser especificada usando nomes abstratos ou uma lista explícita de lugares.
- Os nomes abstratos são *threads*, *cores* e *sockets*, podendo opcionalmente serem seguidos de um número em parenteses, expressando a quantidade de lugares a serem criados.



Variáveis de Ambiente

- Alternativamente, a localização pode ser expressa com lista de lugares separados por vírgula.
- Um lugar é especificado como um conjunto de números não-negativos entre chaves, separados por vírgula, significando as *threads de hardware*, e não cores físicos.
- Para especificar um intervalo, use “:” seguido de uma contagem indicando o comprimento.
- Opcionalmente, o comprimento pode ser seguido por mais um “:” e um número indicando o salto. Se nenhum valor for definido, é assumido “1” como padrão para o salto. Por exemplo:

```
$ setenv OMP_PLACES "{0,1,2}, {3,4,6}, {7,8,9}, {10,11,12}"
```

```
$ setenv OMP_PLACES "{0:3},{3:3}, {7:3}, {10:3}"
```

```
$ setenv OMP_PLACES "{0:2}, {4:3}"
```



Variáveis de Ambiente

- **OMP_PROC_BIND**

- Os valores que essa variável de ambiente pode ter são **true**, **false** ou uma lista separada por vírgulas dos seguintes valores:
 - **master, close ou spread.**
- Se a variável de ambiente for definida como **false**, o ambiente de execução pode mover threads OpenMP entre lugares OpenMP, a afinidade de *thread* é desabilitada e as cláusulas **proc_bind** em construções paralelas são ignoradas.
- Em caso contrário, o ambiente de execução não pode mover as *threads* entre lugares OpenMP, a afinidade de *threads* está ativada e a *thread* inicial é vinculada ao primeiro lugar na lista de lugares OpenMP (como definido pela variável de ambiente OMP_PLACES).



Variáveis de Ambiente

- O comportamento do programa é dependente de implementação, se o valor na variável de ambiente OMP_PROC_BIND não for definido corretamente.
- O comportamento também é dependente de implementação se a thread inicial não puder ser vinculada ao primeiro lugar definido na lista de lugares OpenMP.
- Exemplos de uso são:

```
$ setenv OMP_PROC_BIND false
```

```
$ setenv OMP_PROC_BIND "spread, spread, close"
```

